NEXUS AI

32-Week Detailed Master Build Roadmap

AI Infrastructure • Model Gateway • MCP • Agents • Workflows • Evaluation • Verification • Evidence • Policy • Adaptive Autonomy • Observability • Security

Purpose

This is the detailed week-by-week execution plan for building Nexus as a serious systems project. It preserves the broad original Nexus vision while making the evidence-driven verification and authorization layer the central differentiator. The plan is deliberately structured so that each week produces something usable, testable and explainable before the next layer is added.

Build philosophy

- LEARN → EXPLAIN → DESIGN → BUILD → BREAK → MEASURE → EXPLAIN AGAIN.
- AI is a teacher, reviewer, pair programmer, tester, debugger and interviewer—not the architect.
- Do not copy giant code dumps. Understand interfaces, state, failure modes, security boundaries and tradeoffs.
- Every important subsystem needs tests, metrics and a definition of done.
- Do not prematurely build a giant dashboard, marketplace, graph memory, universal trust score, Kubernetes stack or uncontrolled autonomy.

How to use this roadmap

Weekly execution loop

- Day 1–2: Learn the concepts needed for the week's subsystem.
- Day 2–3: Draw the architecture and write a short design note before coding.
- Day 3–5: Implement the smallest useful vertical slice.
- Day 5–6: Add tests and intentionally break the subsystem.
- Day 6: Measure it and record results.
- Day 7: Explain the subsystem aloud without notes and answer interview questions.

Engineering journal

Maintain NEXUS_ENGINEERING_JOURNAL.md with: What I built; Why; Architecture; Data flow; DB/data structures; Alternatives; Failure cases; Security; Tests; Metrics; Problems; What I still do not understand; Interview questions.

Architecture decision records

- Why FastAPI / PostgreSQL / SQLAlchemy / modular monolith.
- Why provider adapters and a normalized AI gateway.
- Why deterministic routing first and hard constraints before scoring.
- Why evidence is separate from policy.
- Why sensitive actions fail closed.
- Why OpenTelemetry is used for cross-layer observability.
- Why agent state and approvals are persisted.
- Why mature workflow/sandbox infrastructure should be integrated when it provides stronger guarantees than a custom implementation.

Milestones

Weeks	Milestone
1–3	**Foundation**
4–6	**AI Gateway**
7–12	**AI Intelligence**
13–17	**MCP + Evaluation**
18–21	**Agents + Workflows**
22–25	**Verify + Evidence + Policy**
26–29	**Production Platform**
30–32	**Ecosystem + Optimization + Research**

Week 1 — Backend & API Foundation

Turn Nexus from an idea into a clean, testable backend service. The goal is to understand the full request lifecycle before adding AI infrastructure.

1. Learn / Understand

- HTTP request/response lifecycle; methods, headers, status codes, JSON and content types.
- REST resource design, idempotency, validation, pagination basics and error contracts.
- FastAPI routing, dependency injection, Pydantic models, async/await and application lifecycle.
- Configuration through environment variables; structured logging; request IDs; health/readiness concepts.

2. Build — exact implementation scope

- Create the Nexus monorepo and backend application structure.
- Implement /health and /ready endpoints.
- Create versioned API routes such as /api/v1/...
- Add configuration loading and validation.
- Create global error handling with consistent error responses.
- Add request/correlation IDs and structured logs.
- Write the first API/unit tests and a minimal CI check.

3. Architecture / data flow to understand

- Client → HTTP → FastAPI router → dependency validation → service layer → persistence/external adapter → response.
- Keep HTTP concerns in the API layer and business rules in services.
- Do not let provider/database details leak into route handlers.

4. Tests and failure cases

- Malformed JSON/body; invalid types; missing fields; unsupported method; unknown route.
- Internal exception; timeout simulation; dependency failure.
- Test request ID propagation and deterministic error responses.

5. Metrics / evidence

- Request count; error count/rate; request latency p50/p95/p99; health-check success.
- Every request should be traceable by a request ID.

6. Definition of Done

- A fresh machine can start the service from documented commands.
- Health/readiness endpoints work.
- At least one happy-path and several failure-path API tests exist.
- You can explain HTTP → FastAPI → service → response without looking at code.

7. Interview mastery

- Why FastAPI instead of Flask/Django?
- What is async actually buying here?
- What is the difference between 400, 401, 403, 404, 409 and 500?
- Where should validation happen and why?
- What makes an API operation idempotent?

Week 2 — Database + Identity Foundation

Create the persistent control-plane core: users, projects, API keys and ownership boundaries.

1. Learn / Understand

- Relational modeling; primary/foreign keys; constraints; indexes; transactions.
- PostgreSQL connection handling and connection pools.
- SQLAlchemy 2.0 ORM/session patterns and Alembic migrations.
- Password hashing, JWT access tokens, OAuth2 terminology and authorization versus authentication.
- Project/tenant isolation and why object ownership must be checked server-side.

2. Build — exact implementation scope

- Create User, Project, APIKey and membership/role models as appropriate.
- Implement registration and login.
- Hash passwords with a modern password-hashing library.
- Issue and validate JWTs.
- Protect project routes.
- Implement project ownership checks and basic RBAC.
- Create migration history and seed/test data.

3. Architecture / data flow to understand

- User → Project → API key / membership → protected request.
- Authentication answers 'who are you?'; authorization answers 'are you allowed to do this?'
- Never trust project IDs supplied by clients without authorization checks.
- Keep secrets out of plaintext database columns when possible; store only what the architecture needs.

4. Tests and failure cases

- Duplicate email/user; wrong password; expired/invalid token; revoked key.
- Cross-project object access attempt; missing resource; malformed token.
- Transaction rollback test; migration up/down test.

5. Metrics / evidence

- Login success/failure; authorization-denial count; DB query latency; connection-pool health.
- Audit security-sensitive events such as login, key creation and permission changes.

6. Definition of Done

- Registration → login → project creation → protected project retrieval works.
- A user cannot access another user's project.
- Migrations reproduce the schema from an empty database.
- You can explain JWT, sessions, ownership and RBAC.

7. Interview mastery

- JWT versus server-side sessions?
- Where should authorization checks live?
- Why indexes on foreign keys / lookup fields?
- What happens if two requests update the same row concurrently?

• How would you isolate tenants at the database and application layers?

Week 3 — Production Foundation

Make the foundation reproducible and failure-aware rather than a local-only prototype.

1. Learn / Understand

- Docker images/containers, Compose, volumes and networks.
- Environment separation and secret handling.
- pytest unit/integration/API/database testing.
- Readiness versus liveness; startup/shutdown lifecycle.
- Basic distributed-system failure modes: timeout, duplicate request, dependency outage and partial failure.

2. Build — exact implementation scope

- Containerize FastAPI and PostgreSQL.
- Create Docker Compose for local development.
- Add health/readiness checks.
- Create isolated test configuration/database.
- Add unit, API and database integration tests.
- Add formatting/linting and CI.
- Document startup, migrations, tests and common failures.

3. Architecture / data flow to understand

- Browser/client → API container → PostgreSQL container.
- A health endpoint should not necessarily mean the database is ready.
- Configuration must be injectable; code should not contain environment-specific secrets.

4. Tests and failure cases

- Database unavailable; invalid credentials; expired token; duplicate resource.
- Malformed request; migration failure; service restart.
- Run the test suite against a clean database.

5. Metrics / evidence

- Test pass rate; startup success; readiness latency; DB failure recovery time.
- Track flaky tests separately from real product failures.

6. Definition of Done

- One command starts the local stack.
- Tests pass in CI.
- Failure tests demonstrate expected behavior.
- README contains exact run/test/migration instructions.

7. Interview mastery

- What is the difference between liveness and readiness?
- What does Docker isolate and what does it not isolate?
- Why are retries dangerous?
- What does idempotency mean in a distributed service?

Week 4 — AI / Model Gateway

Introduce AI execution behind a provider-independent boundary. This is the first real AI infrastructure layer.

1. Learn / Understand

- LLM API request/response structure; messages; streaming; token usage.
- Provider adapters and interface design.
- Timeouts, retries, backoff and cancellation.
- Synchronous versus asynchronous execution.
- Why normalized internal request/response objects matter.

2. Build — exact implementation scope

- Define a ModelProvider interface.
- Implement one provider adapter first.
- Expose a Nexus chat/completion endpoint.
- Normalize provider responses.
- Support streaming if the chosen provider supports it.
- Record request IDs, model, latency, status and usage.
- Add explicit timeout/retry policy.

3. Architecture / data flow to understand

- Nexus API → AI Gateway → Provider Adapter → Provider API → normalized response.
- Provider-specific SDK code stays behind adapters.
- Retry only failures that are safe to retry; distinguish transport failures from model/application errors.

4. Tests and failure cases

- Provider timeout; HTTP 429; 5xx; malformed provider response; stream interruption.
- Duplicate request; client cancellation; provider unavailable.
- Mock provider tests must not require a real model account.

5. Metrics / evidence

- Latency p50/p95/p99; provider error rate; timeout rate; retry rate; tokens/request; cost/request.
- Record enough metadata to reconstruct why a request succeeded or failed.

6. Definition of Done

- One Nexus endpoint can call a model without exposing provider-specific code.
- Provider failure behavior is tested.
- Usage/latency is recorded.
- You can explain adapter boundaries and retry semantics.

7. Interview mastery

- Why an adapter instead of calling SDKs directly?
- Which errors are retryable?
- How does streaming change resource management?
- Where should token/cost accounting occur?

Week 5 — Multiple Models + Model Registry

Move from one model integration to a real model-control layer.

1. Learn / Understand

- Model capabilities, context windows, pricing and provider health.
- Registry versus hard-coded configuration.
- Fallback strategies and capability constraints.
- Health checks and stale metadata.

2. Build — exact implementation scope

- Add at least two model/provider integrations where practical.
- Create a model registry with model ID, provider, capabilities, context limit, pricing and availability.
- Expose /models or equivalent registry APIs.
- Add provider/model health state.
- Implement basic fallback on eligible provider failures.
- Add model capability validation.

3. Architecture / data flow to understand

- Request → candidate models → capability filtering → selected model → provider adapter.
- Registry is metadata/control data; it should not be confused with the model weights themselves.
- Fallback should preserve the request contract and be observable.

4. Tests and failure cases

- Unavailable model; stale registry entry; incompatible context size; fallback also failing.
- Price metadata missing; unsupported capability requested.
- Test deterministic candidate filtering.

5. Metrics / evidence

- Model availability; fallback rate; provider error rate; model-specific latency and cost.
- Count requests rejected because no candidate satisfied hard constraints.

6. Definition of Done

- At least two providers/models are reachable through the same Nexus interface.
- Model metadata is persisted and queryable.
- Fallback behavior is explicit and tested.

7. Interview mastery

- What belongs in a model registry?
- How do you avoid routing to a model that has gone unhealthy?
- What should happen if all candidates violate a hard constraint?

Week 6 — Usage, Tokens & Cost Control

Make AI execution measurable enough to support routing, budgets and experiments later.

1. Learn / Understand

- Input/output/total token accounting.
- Provider pricing models and why pricing metadata can change.
- Cost estimation versus provider-reported billing.
- Usage events and immutable-ish audit records.

2. Build — exact implementation scope

- Create a usage record containing request ID, project, model, provider, tokens, latency, status, timestamp and estimated cost.
- Build cost aggregation by project/model/provider/time period.
- Add usage APIs and basic budget visibility.
- Handle missing token counts gracefully.

3. Architecture / data flow to understand

- Model request → provider response → usage extractor → normalized usage event → database/analytics store.
- Do not silently treat estimated cost as authoritative billing.
- Keep raw usage metadata so pricing can be recalculated.

4. Tests and failure cases

- Provider returns no usage; malformed usage; price missing; streaming usage incomplete.
- Duplicate event; partial failure after provider success.
- Verify aggregation against hand-calculated examples.

5. Metrics / evidence

- Cost/request; tokens/request; cost/day; provider spend share; latency; retry overhead.
- Track estimated versus reported usage when both exist.

6. Definition of Done

- A project can answer: how many tokens did we use, what did it cost, and which models consumed it?
- Usage is tied to request IDs.
- Aggregation tests pass.

7. Interview mastery

- Why separate usage from request logs?
- How would you handle pricing changes?
- What if a provider succeeds but Nexus crashes before recording usage?

Week 7 — Model Comparison / Benchmark Runner

Build an experiment runner so model choices can be evaluated rather than guessed.

1. Learn / Understand

- Benchmark datasets, test cases, evaluator interfaces and reproducibility.
- Quality versus latency versus cost tradeoffs.
- Deterministic and model-based evaluators.
- Run isolation and experiment versioning.

2. Build — exact implementation scope

- Create benchmark/test-case schemas.
- Implement a runner that sends the same cases to multiple models.
- Store raw outputs plus evaluator results.
- Add exact-match/structured evaluators first.
- Generate comparison reports.
- Version datasets and experiment configurations.

3. Architecture / data flow to understand

- Dataset → test cases → model runner → outputs → evaluator(s) → results → comparison report.
- Keep raw outputs; never store only a scalar score.
- Separate model execution from evaluation.

4. Tests and failure cases

- Provider failure; evaluator failure; malformed structured output; timeout.
- Non-deterministic model output; duplicate run.
- Repeat a subset to estimate variance.

5. Metrics / evidence

- Quality score; pass rate; latency; cost; tokens; failure rate.
- Confidence intervals or repeated-run variation where appropriate.

6. Definition of Done

- Same benchmark can be run across models and reproduced later.
- Report shows quality/cost/latency together.
- You can defend why a model was selected.

7. Interview mastery

- How do you compare two stochastic models fairly?
- When is an LLM judge acceptable?
- Why store raw outputs?

Week 8 — Intelligent Model Routing

Turn model comparison data into controlled routing decisions.

1. Learn / Understand

- Hard constraints versus soft scoring.
- Weighted scoring and normalization.
- Routing policies, fallbacks and deterministic baselines.
- Cold-start and missing-metric problems.

2. Build — exact implementation scope

- Implement hard filters: context, capability, availability and budget.
- Implement deterministic scoring across quality, cost, latency and capability.
- Record the routing decision and candidate set.
- Add a fixed-model baseline for comparison.
- Expose routing configuration and reason codes.

3. Architecture / data flow to understand

- Request → hard constraints → eligible candidates → score → selected model → execution → routing record.
- Never let a soft score override a hard safety/capability constraint.
- The decision should be inspectable.

4. Tests and failure cases

- No eligible model; stale metrics; tie scores; missing quality data.
- Router chooses expensive model under budget; provider fails after selection.
- Test every hard constraint independently.

5. Metrics / evidence

- Routing accuracy against benchmark objective; cost reduction; latency; quality regression; fallback rate.
- Measure decision distribution across models.

6. Definition of Done

- Every request has a routing decision with reasons.
- A fixed baseline can be compared against routing.
- Routing does not violate hard constraints.

7. Interview mastery

- Why hard constraints first?
- How would you handle missing metrics?
- How do you prevent the router from optimizing cost at the expense of quality?

Week 9 — Token & Context Optimization

Reduce token/cost/latency while measuring whether optimization damages task quality.

1. Learn / Understand

- Context windows and token budgets.
- Message compaction, summarization and redundancy.
- Retrieval filtering and relevance.
- Output limits and quality regression.

2. Build — exact implementation scope

- Add context budgets per request/project.
- Implement message compaction for long histories.
- Add simple redundant-context detection.
- Add retrieval/context filtering hooks.
- Control output limits.
- Run baseline versus optimized benchmark experiments.

3. Architecture / data flow to understand

- Raw conversation/retrieval → context planner → compact/filter/summarize → model request.
- Optimization is a transformation; preserve enough metadata to compare against the original.
- Do not optimize tokens blindly.

4. Tests and failure cases

- Summarizer drops required information; context exceeds limit; optimization increases latency.
- Quality regression; malformed prompt after compaction.
- Golden cases for important context-preservation behaviors.

5. Metrics / evidence

- Tokens saved; cost saved; latency change; quality delta; failure/regression rate.
- Track optimization overhead separately.

6. Definition of Done

- A benchmark demonstrates a measurable token reduction with an explicit quality bound.
- You can explain when NOT to optimize aggressively.

7. Interview mastery

- How do you prove token reduction did not harm quality?
- What information should never be removed?
- When could summarization be worse than sending the original context?

Week 10 — Semantic Cache

Introduce caching only as a measurable optimization, not as a feature checkbox.

1. Learn / Understand

- Exact caching versus semantic caching.
- Embeddings/similarity and false-positive cache hits.
- TTL, invalidation and project isolation.
- Cache correctness and quality regression.

2. Build — exact implementation scope

- Implement exact request cache first.
- Add semantic similarity as a separate layer.
- Add TTL and invalidation.
- Namespace cache entries by project and relevant configuration.
- Store model/prompt/version metadata.
- Benchmark no-cache versus cache.

3. Architecture / data flow to understand

- Request → exact/semantic cache lookup → hit or model execution → cache write.
- Cache keys must include semantics that affect output.
- Semantic cache should have conservative thresholds and observability.

4. Tests and failure cases

- Wrong semantic match; stale prompt/model; privacy leak across projects; invalidation failure.
- Embedding service failure; cache stampede.
- Test cross-project isolation and stale-entry behavior.

5. Metrics / evidence

- Hit rate; incorrect-cache rate; latency reduction; cost savings; cache lookup overhead.
- Quality delta between cached and uncached responses.

6. Definition of Done

- Cache demonstrably reduces cost/latency without unacceptable correctness loss.
- Isolation and invalidation are tested.

7. Interview mastery

- Why can a high cache hit rate still be bad?
- How would you prevent cross-tenant leakage?
- What should invalidate a cached response?

Week 11 — Prompt Registry & Versioning

Make prompts first-class, versioned engineering artifacts.

1. Learn / Understand

- Prompt versioning, variables, compatibility and reproducibility.
- Rollback and evaluation of prompt changes.
- Why prompt text alone is insufficient for reproducibility.

2. Build — exact implementation scope

- Create prompt and prompt-version tables.
- Support variables and validation.
- Attach model compatibility and evaluation results.
- Implement active version and rollback.
- Link prompt version to every AI request/experiment.

3. Architecture / data flow to understand

- Developer → prompt registry → resolved version → model gateway.
- A result should identify prompt version, model version/ID and relevant runtime configuration.

4. Tests and failure cases

- Missing variable; incompatible version; deleted active prompt; rollback race.
- Prompt injection through variable content; unsafe template composition.
- Version integrity tests.

5. Metrics / evidence

- Prompt success/pass rate; version-to-version quality delta; rollback count.
- Track prompt version in traces and experiment records.

6. Definition of Done

- You can answer exactly which prompt version produced a result.
- A prompt can be rolled back without code redeployment.
- Prompt changes can be evaluated.

7. Interview mastery

- Why version prompts separately from source code?
- How do you reproduce a model result six months later?

Week 12 — Cost, Quotas & Hierarchical Budgets

Turn usage data into enforceable financial and operational controls.

1. Learn / Understand

- Rate limits, quotas, budgets and concurrency limits.
- Hard versus soft limits.
- Hierarchical accounting: organization → project → agent → user → request.
- Race conditions in quota enforcement.

2. Build — exact implementation scope

- Implement token and monetary budgets.
- Add project-level and request-level limits.
- Add rate limiting and concurrency controls.
- Create alerts for soft thresholds.
- Implement hard rejection for configured limits.
- Record budget decisions and remaining capacity.

3. Architecture / data flow to understand

- Request → identity/project context → budget check → reserve/execute → finalize usage.
- Reservation may be needed before expensive execution to avoid overspending.
- Budget checks must be atomic enough for concurrent requests.

4. Tests and failure cases

- Concurrent overspend; stale counters; provider cost mismatch; retry double-counting.
- Rate-limit burst behavior; partial execution.
- Stress test concurrent budget updates.

5. Metrics / evidence

- Budget utilization; rejection rate; overspend; rate-limit events; concurrency queue time.

6. Definition of Done

- Budgets are enforced under concurrency.
- The system can explain why a request was rejected.
- Retries do not silently double-charge internal accounting.

7. Interview mastery

- How do you make quota enforcement race-safe?
- Hard versus soft budget: when would you use each?
- What happens if actual provider cost differs from estimate?

Week 13 — MCP Fundamentals

Understand MCP deeply enough to build the Nexus MCP layer rather than treating MCP as magic.

1. Learn / Understand

- MCP concepts: tools, resources, prompts, schemas and lifecycle.
- Client/server roles, discovery, invocation and transport concepts.
- Authorization and trust boundaries.
- Schema validation and tool argument safety.

2. Build — exact implementation scope

- Build a tiny deterministic MCP server.
- Build a tiny client that discovers and invokes one tool.
- Add structured schemas.
- Test invalid arguments, timeout and tool failure.
- Document the complete request lifecycle.

3. Architecture / data flow to understand

- Nexus client → MCP server → tool implementation → result → Nexus.
- MCP protocol concerns should remain separate from Nexus policy concerns.
- Tool schemas are an input-validation boundary, not proof of safety.

4. Tests and failure cases

- Invalid arguments; server unavailable; timeout; malformed result; malicious/overly broad tool behavior.
- Connection lifecycle failure and retry behavior.

5. Metrics / evidence

- Tool-call latency; success/failure rate; timeout rate; invalid-argument rate.

6. Definition of Done

- You can implement and explain a tiny MCP server/client without a tutorial.
- You can distinguish protocol mechanics from authorization policy.

7. Interview mastery

- What is MCP solving?
- What does discovery mean?
- Why is schema validation insufficient for authorization?

Week 14 — Nexus MCP Client Layer

Turn MCP knowledge into a reusable client subsystem.

1. Learn / Understand

- Connection management, discovery, tool schemas and invocation.
- Timeouts, retries and connection lifecycle.
- Normalization of tool calls/results.
- Per-server credentials and configuration.

2. Build — exact implementation scope

- Create MCP connection records.
- Implement discovery and tool listing.
- Normalize tool metadata.
- Implement tool invocation with timeout.
- Persist tool-call metadata.
- Add connection health state.

3. Architecture / data flow to understand

- Agent/application → Nexus MCP client → selected server/tool → result → execution record.
- The client layer should not decide business authorization on its own.

4. Tests and failure cases

- Server unavailable; invalid schema; timeout; malformed output; connection reset.
- Retry-safe versus non-retry-safe tool calls.

5. Metrics / evidence

- Discovery latency; tool success rate; timeout rate; connection failures; tool-call volume.

6. Definition of Done

- Nexus can connect to a local MCP server, discover tools and invoke one reliably.
- Failures are observable and bounded.

7. Interview mastery

- Where do MCP credentials live?
- How do you know whether a tool call is safe to retry?
- What information must be recorded for replay?

Week 15 — MCP Gateway + Tool Registry

Create the controlled tool-access boundary where Nexus starts becoming a real control plane.

1. Learn / Understand

- Tool registries, schemas, permissions, allowlists/denylists.
- Identity propagation and authorization boundaries.
- Tool health, rate limits and telemetry.
- Why evidence-aware authorization belongs above raw MCP protocol.

2. Build — exact implementation scope

- Create server/tool registry models.
- Register servers, tools, schemas, capabilities, owners and versions.
- Add argument validation.
- Add tool allowlists/denylists and permissions.
- Add rate limits and per-tool telemetry.
- Expose discovery through Nexus APIs.
- Record every tool invocation with identity and policy context.

3. Architecture / data flow to understand

- Agent/request → MCP gateway → identity → policy/permission check → schema validation → MCP client → server/tool.
- The gateway is the interception point for future evidence-conditioned authorization.
- Keep protocol errors distinct from policy denials.

4. Tests and failure cases

- Unauthorized tool; malicious arguments; server poisoning; SSRF-like network behavior; timeout; excessive calls.
- Cross-project tool access; stale tool version.

5. Metrics / evidence

- Allowed/denied tool calls; invalid args; latency; failures; rate-limit events; policy decision distribution.

6. Definition of Done

- Tool access is centralized and auditable.
- A denied call never reaches the external tool.
- Every call has enough metadata for later replay/investigation.

7. Interview mastery

- Where should authorization happen?
- What is the difference between tool discovery and permission to use the tool?
- How would you defend against a malicious MCP server?

Week 16 — Evaluation Engine

Build a general evaluation layer that can score models, agents and eventually verification systems.

1. Learn / Understand

- Datasets, cases, evaluator versions and experiment configuration.
- Deterministic evaluators versus LLM judges.
- Regression detection and reproducibility.
- Raw artifacts versus derived scores.

2. Build — exact implementation scope

- Create dataset, case, evaluator and result models.
- Implement exact-match and structured-validation evaluators.
- Add code-test evaluator hooks.
- Add optional LLM-judge evaluator with explicit uncertainty.
- Store evaluator version and raw evidence.
- Create regression comparisons.

3. Architecture / data flow to understand

- Dataset → execution → output → evaluator(s) → evidence/results → experiment report.
- Keep evaluator logic versioned.
- Do not collapse all evidence into one irreversible scalar.

4. Tests and failure cases

- Evaluator crash; ambiguous output; stochastic judge; contradictory evaluators.
- False positive/negative regression detection.

5. Metrics / evidence

- Pass rate; evaluator disagreement; regression rate; evaluator latency/cost.
- For judges, track agreement with deterministic ground truth where available.

6. Definition of Done

- A benchmark can be rerun with versioned dataset/evaluator/config.
- Evaluation results are inspectable rather than just a dashboard score.

7. Interview mastery

- When is an LLM judge appropriate?
- How do you detect evaluator drift?
- How do you handle contradictory evidence?

Week 17 — Experiment Platform + Research Methodology

Make Nexus scientifically testable instead of a collection of features.

1. Learn / Understand

- Controlled experiments, baselines, ablations and confounders.
- Repeated trials, variance and confidence intervals.
- Metrics: quality, cost, latency, safety, intervention and failure rates.
- Reproducibility and experiment manifests.

2. Build — exact implementation scope

- Implement experiment definitions and run manifests.
- Add baseline comparison.
- Run: fixed model vs routing.
- Run: baseline vs token optimization.
- Run: no cache vs semantic cache.
- Run: static permissions vs evidence-based authorization.
- Run: LLM-only vs deterministic vs hybrid verification.
- Run: agent with vs without Nexus.

3. Architecture / data flow to understand

- Experiment config → versioned inputs → execution runs → raw artifacts → evaluators → statistical comparison.
- Do not compare systems with different workloads without controlling the dataset/task distribution.

4. Tests and failure cases

- Flaky benchmark; sample leakage; model/provider change mid-experiment.
- Confounded cost/quality results; insufficient sample size.

5. Metrics / evidence

- Task success; quality; cost; latency; unsafe tool calls; unnecessary calls; human intervention; recovery rate.
- Confidence intervals and effect sizes where practical.

6. Definition of Done

- You have at least 4–6 reproducible benchmark families.
- A reviewer can inspect how each metric was produced.
- Claims in the README are backed by experiment data.

7. Interview mastery

- What is the baseline?
- What would invalidate your hypothesis?
- How do you distinguish statistical noise from a real improvement?

Week 18 — Agent Runtime

Introduce stateful agents as explicit systems, not hidden loops inside one Python function.

1. Learn / Understand

- Agent state machines, run identity and checkpoints.
- Planning/execution/tool calling.
- Persisted state versus in-memory state.
- Cancellation and recovery semantics.

2. Build — exact implementation scope

- Create agent, run, message, tool-call and checkpoint models.
- Implement explicit states: CREATED, PLANNING, WAITING_TOOL, EXECUTING, OBSERVING, COMPLETED, FAILED, WAITING_APPROVAL, PAUSED, CANCELLED.
- Persist state transitions.
- Implement a basic planner/executor loop.
- Record model and tool calls.

3. Architecture / data flow to understand

- Agent request → run creation → planning → tool selection → policy → tool call → observation → next state.
- State transitions must be explicit and recoverable.
- Do not depend on a Python process surviving for correctness.

4. Tests and failure cases

- Process crash mid-run; duplicate tool call; invalid transition; stuck run.
- Model timeout; tool failure; cancellation during execution.

5. Metrics / evidence

- Task success; run duration; failure rate; state-transition errors; tool-call count; recovery rate.

6. Definition of Done

- A run can be inspected after process restart.
- State machine transitions are validated.
- Basic agent can complete a multi-step task.

7. Interview mastery

- Why persist state?
- What happens after a worker crashes?
- How do you prevent duplicate side effects?

Week 19 — Agent Tools, Retries, Checkpoints & Approvals

Make agent execution operationally safe before adding more autonomy.

1. Learn / Understand

- Retry semantics, idempotency and side effects.
- Checkpointing, pause/resume and cancellation.
- Human-in-the-loop approval as a first-class state.
- Tool call budgets and execution limits.

2. Build — exact implementation scope

- Implement retry policies by error class.
- Add checkpoints before/after important side effects.
- Implement WAITING_APPROVAL and resume.
- Add cancellation and pause/resume.
- Attach idempotency keys to retry-sensitive actions.
- Add tool-call and time budgets.

3. Architecture / data flow to understand

- Run state → checkpoint → policy/approval → side effect → checkpoint → next state.
- Approval must be represented in persisted state, not a boolean in memory.
- Retries must be aware of whether a side effect may already have occurred.

4. Tests and failure cases

- Double execution; approval race; resume after stale state; cancellation during side effect.
- Retry storm and budget exhaustion.

5. Metrics / evidence

- Retry success rate; duplicate-side-effect rate; approval wait time; human intervention rate; recovery rate.

6. Definition of Done

- A failed worker can resume from a checkpoint.
- Sensitive actions pause and require approval.
- Retries are bounded and observable.

7. Interview mastery

- What is an idempotency key?
- How would you retry a payment-like action safely?
- Why is approval a state rather than a flag?

Week 20 — Memory Layer

Add memory only after the runtime can execute reliably.

1. Learn / Understand

- Session memory versus long-term memory.
- Retrieval, embeddings and semantic search.
- Memory relevance, staleness and deletion.
- Why graph memory is postponed.

2. Build — exact implementation scope

- Implement session memory first.
- Add persistent memory records.
- Add semantic retrieval only when a concrete use case exists.
- Define memory scopes and ownership.
- Add memory provenance and timestamps.
- Evaluate retrieval relevance.

3. Architecture / data flow to understand

- Agent → memory manager → storage/retrieval → selected memories → context builder → model.
- Memory retrieval is a decision that affects model behavior; record what was retrieved.
- Avoid storing secrets or unbounded context blindly.

4. Tests and failure cases

- Irrelevant memory; stale memory; cross-tenant leakage; deletion failure.
- Embedding outage; retrieval timeout.

5. Metrics / evidence

- Retrieval precision/recall proxy; useful-memory rate; context tokens added; stale-memory rate.

6. Definition of Done

- Agent can remember a useful project/session fact across runs.
- Memory is isolated and traceable.
- You can explain why a memory was retrieved.

7. Interview mastery

- Why not use a vector DB immediately?
- How do you delete a user's memory?
- How does memory affect reproducibility?

Week 21 — Workflow Engine

Coordinate deterministic multi-step work with explicit execution semantics.

1. Learn / Understand

- DAGs/state machines; sequential and parallel execution.
- Conditions, timeouts, retries and compensation.
- Workflow state persistence.
- Difference between an agent deciding steps and a workflow enforcing steps.

2. Build — exact implementation scope

- Implement sequential workflow A→B→C.
- Implement parallel branches A→(B,C)→D.
- Add conditional step.
- Add timeout/retry.
- Add approval step.
- Persist workflow state and step results.
- Keep compensation hooks for later.

3. Architecture / data flow to understand

- Workflow definition → run → scheduler/executor → step state → outputs → next eligible steps.
- Agent may propose a plan; workflow engine should enforce declared execution semantics.

4. Tests and failure cases

- Worker crash; dependency failure; branch partial success; timeout; duplicate step execution.
- Invalid DAG/cycle; compensation failure.

5. Metrics / evidence

- Workflow completion rate; step latency; retry count; blocked runs; parallel speedup.

6. Definition of Done

- A workflow survives process restart.
- Parallel and conditional semantics are testable.
- You can explain why a durable workflow product may eventually be preferable to a custom engine.

7. Interview mastery

- What makes a workflow durable?
- How do you prevent duplicate step execution?
- When should Nexus integrate Temporal or another mature engine instead of building more?

Week 22 — Nexus Verify — Verification Engine

Build the major differentiated subsystem: evidence-driven verification of AI-generated code/actions.

1. Learn / Understand

- Sandboxing, deterministic tests and static analysis.
- Correctness, security and performance evidence.
- Fuzzing, property-based, mutation, differential and metamorphic testing concepts.
- Why LLM review is a signal rather than proof.

2. Build — exact implementation scope

- Start narrowly with AI-generated Python changes.
- Accept a patch/code artifact plus test suite.
- Run in an isolated sandbox.
- Execute deterministic tests.
- Run static/security checks.
- Collect logs/artifacts/resource results.
- Return PASS / FAIL / REVIEW with reasons.
- Keep the pipeline extensible for fuzzing and other evidence later.

3. Architecture / data flow to understand

- Proposed change → sandbox → tests/static/security/resource checks → evidence records → verification result.
- The sandbox is a hard security boundary and must not share secrets by default.
- Deterministic evidence should remain distinguishable from LLM judgment.

4. Tests and failure cases

- Sandbox escape; network exfiltration; fork bombs/resource exhaustion; malicious dependencies.
- Flaky tests; tool failure; partial verification; false positive/negative.
- Run adversarial fixtures intentionally.

5. Metrics / evidence

- Pass rate; hidden-test pass rate; security findings; false approval rate; false rejection rate; verification latency; cost.
- Evidence coverage by verification layer.

6. Definition of Done

- A code change can be verified in isolation.
- Evidence is stored with provenance.
- Sensitive actions can depend on verification outcome.
- Known limitations are documented.

7. Interview mastery

- What does 'verified' actually mean?
- How would you prevent sandbox escape?
- Which checks are independent?
- Why is 'LLM says code is safe' insufficient?

Week 23 — Evidence Ledger + Provenance

Make Nexus decisions reconstructible by preserving the evidence behind them.

1. Learn / Understand

- Evidence as an architectural primitive.
- Provenance, freshness, confidence, severity and artifact references.
- Fact → inference → decision separation.
- Append-only/event-like evidence design and contradiction handling.

2. Build — exact implementation scope

- Create evidence records with type, source, timestamp, artifact reference, provenance, freshness, confidence/severity and subject.
- Link evidence to verification runs, tool calls, model outputs and policies.
- Support contradictory evidence instead of overwriting it.
- Build an evidence timeline for a run.
- Store artifacts separately from metadata where appropriate.

3. Architecture / data flow to understand

- FACT: unit tests passed → INFERENCE: patch likely correct → DECISION: permit action.
- Original facts remain even if later interpretation changes.
- Evidence should identify its source and generation method.

4. Tests and failure cases

- Stale evidence; duplicated evidence; conflicting findings; missing provenance.
- Artifact deleted while evidence remains; clock/time inconsistencies.

5. Metrics / evidence

- Evidence freshness; provenance completeness; contradiction count; artifact availability; decision reconstruction success rate.

6. Definition of Done

- Given a decision ID, you can reconstruct what facts/evidence led to it.
- Facts are not silently replaced by model opinions.
- Evidence records are queryable by run/action.

7. Interview mastery

- What makes evidence independent?
- How should contradictory evidence be handled?
- Why not store only the final trust score?

Week 24 — Policy Engine

Turn context and evidence into deterministic, versioned authorization decisions.

1. Learn / Understand

- Policy evaluation, precedence and deny overrides.
- Risk levels and action/resource classification.
- Policy versioning, dry-run and testing.
- ALLOW / BLOCK / APPROVAL_REQUIRED semantics.

2. Build — exact implementation scope

- Define an action schema: agent, tool, action, resource, environment and risk.
- Create versioned policies.
- Implement allow/block/approval decisions.
- Add explicit deny overrides.
- Add dry-run evaluation.
- Create policy test fixtures.
- Store policy version and decision inputs.

3. Architecture / data flow to understand

- Proposed action → identity/context → policy evaluation → decision → optional approval → execution.
- Policy is deterministic; evidence is an input, not the policy itself.
- Policy decisions must be replayable.

4. Tests and failure cases

- Policy conflict; missing context; stale policy; fail-open bug.
- Incorrect resource matching; privilege escalation.

5. Metrics / evidence

- Allow/block/approval distribution; policy evaluation latency; policy-test pass rate; overrides; replay consistency.

6. Definition of Done

- Every sensitive action gets a deterministic policy decision.
- Policy versions are immutable/referencable.
- Failure defaults are explicitly chosen, generally fail-closed for sensitive actions.

7. Interview mastery

- Why separate policy and evidence?
- What does fail closed mean here?
- How do policy precedence and deny rules work?

Week 25 — Evidence-Conditioned Authorization + Scoped Adaptive Autonomy

Create Nexus's central thesis: evidence can change how much autonomy an agent is allowed to exercise, within explicit bounds.

1. Learn / Understand

- Scoped reliability profiles instead of universal trust scores.
- Minimum sample sizes, uncertainty and confidence intervals.
- Risk-weighted authority.
- Evidence freshness and distribution shift.
- Bounded autonomy and rollback.

2. Build — exact implementation scope

- Define proposed-action risk tiers.
- Define reliability history per agent/tool/action/resource class.
- Require minimum evidence before granting broader autonomy.
- Use evidence freshness and uncertainty in decisions.
- Implement bounded transitions such as approval-required → autonomous for a narrow low-risk action class.
- Add rollback/automatic downgrade on failures.
- Do not create a single universal 'trust score'.

3. Architecture / data flow to understand

- Action proposal → risk classification → current policy → evidence query → scoped reliability profile → authorization decision.
- Authority is scoped by action/risk/resource, not globally inherited.
- New or uncertain agents default to conservative permissions.

4. Tests and failure cases

- Rare catastrophic failure hidden by averages; distribution shift; feedback loops; gaming evidence.
- Correlated evidence sources; stale success history; insufficient sample size.

5. Metrics / evidence

- Unsafe-action rate; false-approval rate; human intervention; autonomy rate; recovery rate; confidence intervals; rollback/downgrade count.

6. Definition of Done

- You can demonstrate a measurable safety/efficiency tradeoff.
- Autonomy increases only for explicitly scoped actions.
- Uncertainty and minimum sample requirements are enforced.
- A failure can automatically reduce authority.

7. Interview mastery

- Why not one trust score?
- What evidence should increase or decrease authority?
- How do you avoid a feedback loop where autonomy generates the evidence that justifies more autonomy?
- What happens under distribution shift?

Week 26 — Observability + Replay

Make every important Nexus decision inspectable and reproducible.

1. Learn / Understand

- OpenTelemetry concepts: traces, spans, attributes, propagation.
- Metrics/logs versus traces.
- Run timelines and causal reconstruction.
- Replay without blindly rerunning stochastic model calls.

2. Build — exact implementation scope

- Instrument model, MCP, agent, workflow, verification and policy operations.
- Link request ID, trace ID, run ID and decision ID.
- Create an event timeline.
- Implement replay that reconstructs prior state/evidence/configuration.
- Add decision comparison: old policy versus candidate policy on historical runs.
- Redact sensitive values.

3. Architecture / data flow to understand

- Client → API trace → model span → tool span → verification spans → policy decision → action.
- Replay should be explicit about which components are reused and which are rerun.
- Do not log secrets or full sensitive prompts indiscriminately.

4. Tests and failure cases

- Missing spans; cardinality explosion; PII leakage; replay divergence.
- Trace sampling hides important failure paths.

5. Metrics / evidence

- Trace completeness; span latency; replay reconstruction success; missing-context rate; policy-diff impact.

6. Definition of Done

- A reviewer can inspect a failed run end-to-end.
- A historical decision can be replayed against a new policy without executing the original side effect.

7. Interview mastery

- Why OpenTelemetry?
- What is the difference between a log and a trace?
- How would you replay a stochastic agent safely?

Week 27 — Security Architecture + Threat Modeling

Treat security as architecture, not a final checklist.

1. Learn / Understand

- Authentication/authorization boundaries.
- Prompt injection, tool poisoning and malicious MCP servers.
- SSRF, sandbox escape, secret exfiltration, privilege escalation.
- Supply-chain risk and dependency security.
- Tenant isolation, rate limits, auditability.

2. Build — exact implementation scope

- Create a Nexus threat model.
- Map trust boundaries and attack surfaces.
- Add secret redaction.
- Harden tool/network access.
- Add sandbox resource limits.
- Add dependency/security scanning.
- Create security regression tests.
- Document fail-closed behavior for sensitive operations.

3. Architecture / data flow to understand

- Untrusted AI output → proposed action → policy → tool/sandbox boundary.
- External tools and model outputs are untrusted inputs.
- The verification sandbox must be separated from production credentials/network.

4. Tests and failure cases

- Prompt injection fixture; malicious tool arguments; SSRF attempt; secret-leak attempt; oversized payload; privilege escalation.
- Cross-tenant data access; replay attack.

5. Metrics / evidence

- Security test pass rate; blocked attack attempts; secret-redaction coverage; unauthorized-action rate; sandbox violations.

6. Definition of Done

- A threat model exists and maps to controls/tests.
- Sensitive boundaries are explicit.
- Security failures are observable and fail safely.

7. Interview mastery

- Where is your trust boundary?
- How could an agent cause SSRF?
- How do you stop a tool from exfiltrating credentials?
- What should happen when a security check itself fails?

Week 28 — SDK + CLI + Webhooks + Nexus MCP Server

Turn Nexus into a platform another developer can actually use.

1. Learn / Understand

- SDK design, API stability and typed clients.
- CLI command design and authentication.
- Webhooks, signatures, retries and idempotency.
- Nexus as an MCP server exposing selected capabilities.

2. Build — exact implementation scope

- Build Python SDK first.
- Add CLI commands for projects, models, benchmarks, runs, verification, policies and logs.
- Add signed webhooks for run completion, approvals, verification and budget events.
- Implement webhook retry/idempotency.
- Expose selected Nexus capabilities through a Nexus MCP server: run_agent, search_memory, run_workflow, evaluate_code, get_agent_status, usage/cost.

3. Architecture / data flow to understand

- SDK/CLI → Nexus API → control plane.
- External AI client → Nexus MCP server → Nexus authorization → internal capability.
- The MCP server must not bypass normal Nexus authorization.

4. Tests and failure cases

- Invalid webhook signature; duplicate delivery; expired credentials; incompatible SDK version.
- MCP client attempts unauthorized capability.

5. Metrics / evidence

- SDK request success; webhook delivery success; duplicate-safe delivery rate; CLI command failure; MCP authorization denials.

6. Definition of Done

- A new developer can operate core Nexus functions without reading internal source.
- Webhooks are signed and retry-safe.
- Nexus MCP server respects policy boundaries.

7. Interview mastery

- How do you version an SDK?
- How do webhook signatures prevent spoofing?
- Why expose Nexus through MCP?

Week 29 — Dashboard + Developer Experience

Build a UI that exposes engineering truth rather than hiding it behind decorative charts.

1. Learn / Understand

- Information architecture for infrastructure systems.
- Drill-down UX: KPI → request/run → evidence/config/version.
- Operational dashboards versus product analytics.

2. Build — exact implementation scope

- Create views for projects, models, requests, cost, tokens, routing, agents, tools, workflows, evaluations, verification, evidence, policies, approvals, traces and replay.
- Ensure every KPI links to underlying records.
- Add filters by project, model, agent, time, policy and run.
- Show verification evidence and policy decision explanations.

3. Architecture / data flow to understand

- Dashboard → APIs → persisted control-plane records.
- The UI should be a view over real telemetry/evidence, not a separate truth source.

4. Tests and failure cases

- Pagination failure; stale data; missing permissions; expensive queries.
- Cross-tenant visibility bugs.

5. Metrics / evidence

- Dashboard query latency; error rate; drill-down completion; stale-data indicators.

6. Definition of Done

- A reviewer can follow one request from cost → model → tool → verification → policy → final decision.
- Permissions are enforced at API level, not only UI level.

7. Interview mastery

- Why is UI authorization insufficient?
- How would you optimize a dashboard query over millions of traces?
- What information is essential for debugging an AI run?

Week 30 — Plugins, Marketplace & Multi-Agent

Only after the core control plane works, add ecosystem and multi-agent capabilities.

1. Learn / Understand

- Plugin contracts, capabilities and version compatibility.
- Permission disclosure and signing.
- Multi-agent delegation and authority propagation.
- Why multi-agent systems should not inherit unlimited authority.

2. Build — exact implementation scope

- Define plugin manifest: identity, capabilities, version, permissions, config schema, health.
- Support signed/versioned plugin metadata.
- Design marketplace entries for MCP servers/tools/prompts/eval packs/integrations.
- Add bounded multi-agent delegation with explicit capabilities and context.
- Propagate policy constraints to delegated agents.

3. Architecture / data flow to understand

- Parent agent → delegation request → capability subset → child agent → tool/policy checks.
- Delegation must not automatically increase authority.
- Plugin installation is a supply-chain/security boundary.

4. Tests and failure cases

- Malicious plugin; capability escalation; incompatible version; compromised dependency.
- Child agent bypasses parent constraints.

5. Metrics / evidence

- Plugin install failures; unauthorized capability requests; delegation violations; multi-agent task success.

6. Definition of Done

- A plugin cannot silently gain permissions.
- Delegated agents have bounded capabilities.
- Marketplace remains secondary to the core thesis.

7. Interview mastery

- Why is a marketplace risky?
- How should authority propagate across agents?
- How do you revoke a compromised plugin?

Week 31 — Self-Optimization + AI DevOps

Let Nexus propose improvements while preserving human/policy control over production changes.

1. Learn / Understand

- Optimization loops, offline evaluation and rollout safety.
- Distribution shift, feedback loops and Goodhart's law.
- Canary rollout, rollback and change proposals.

2. Build — exact implementation scope

- Generate proposals for routing weights, prompt versions, cache thresholds, budgets or policies.
- Evaluate proposals offline against benchmark datasets.
- Require policy review.
- Roll out only to a limited scope.
- Monitor outcomes and automatically rollback unsafe regressions.
- Store proposal provenance and experiment evidence.

3. Architecture / data flow to understand

- Observe → propose → benchmark offline → policy review → limited rollout → monitor → rollback.
- AI may recommend configuration changes; deterministic controls decide whether they can be applied.

4. Tests and failure cases

- Metric gaming; feedback loop; distribution shift; regression hidden by aggregate metrics.
- Bad policy rollout; inability to rollback.

5. Metrics / evidence

- Quality delta; cost delta; latency delta; safety regression; rollback rate; proposal acceptance rate.

6. Definition of Done

- No autonomous production self-modification without evidence and explicit controls.
- A proposal can be reproduced and rolled back.
- Optimization is measurable.

7. Interview mastery

- What is Goodhart's law here?
- Why offline evaluation before rollout?
- How do you prevent the system from optimizing a proxy metric while making the product worse?

Week 32 — Productionization + Chaos + Research + Portfolio Demo

Turn the project into a reproducible engineering/research artifact that can survive skeptical technical review.

1. Learn / Understand

- Scaling, workers, queues/events, backpressure and resource limits.
- Backup/recovery, deployment and runbooks.
- Chaos/failure engineering.
- Benchmark protocol, ablations, confidence intervals and limitations.
- How to present architecture and evidence in an interview.

2. Build — exact implementation scope

- Load/performance test critical paths.
- Introduce controlled failures: provider outage, DB outage, MCP timeout, worker crash, policy failure, verification failure.
- Add backup/recovery procedures.
- Document deployment and operational runbooks.
- Finalize benchmark suite and ablation studies.
- Create a portfolio demo: agent proposes action/code change → risk/policy → verification → evidence → ALLOW/BLOCK/APPROVAL_REQUIRED → optional repair/execution → replay.
- Publish architecture, metrics, limitations and reproducibility instructions.

3. Architecture / data flow to understand

- Production architecture should evolve only when a measured bottleneck requires it.
- Use queues/events where asynchronous workloads justify them.
- Keep the evidence/policy/replay chain central to the final demo.

4. Tests and failure cases

- Chaos test every major dependency.
- Verify recovery from worker/process failure.
- Test backup restoration.
- Test policy fail-closed behavior.
- Measure verification false positives/negatives.

5. Metrics / evidence

- Availability; p95/p99 latency; throughput; recovery time; error budget; unsafe-action rate; false approval/rejection; cost; benchmark effect sizes.

6. Definition of Done

- A clean environment can reproduce the demo.
- Architecture and tradeoffs are documented.
- At least one strong benchmark demonstrates Nexus's central value.
- Known limitations and failure cases are explicitly stated.
- You can defend every major component in an interview.

7. Interview mastery

- Why did you build Nexus this way?
- What did you deliberately NOT build?
- What is the strongest evidence that Nexus improves AI reliability?
- Where can Nexus still fail?
- If traffic grows 100×, what changes first?
- Which component is the hardest security boundary?
- What part could become a research paper?

Final Nexus Definition of Done

- A developer can authenticate, create a project and securely call models through a unified Nexus API.
- Nexus can compare models and route requests using explicit constraints and measurable objectives.
- Token/context optimization and caching have benchmark evidence rather than anecdotal claims.
- MCP tools are registered, validated, authorized, rate-limited and observable.
- Agents have persisted state, checkpoints, retries, approvals and recovery semantics.
- Workflows have explicit sequential/parallel/conditional behavior and survive process failure.
- Nexus Verify can independently test and analyze an AI-generated Python change in a constrained environment.
- Evidence is provenance-bearing and separated into facts, inferences and decisions.
- Policy decisions are deterministic, versioned and replayable.
- Adaptive autonomy is scoped to action/resource/risk classes and is backed by sufficient, fresh evidence.
- Observability connects requests, model calls, tools, agents, verification, policy and final actions.
- Security boundaries, threat models and failure behavior are documented and tested.
- The final demo proves a measurable reliability/safety advantage rather than merely showing a dashboard.

The central Nexus thesis

Nexus should not be presented as 'another AI gateway' or 'an LLM that judges another LLM'. The stronger thesis is: Nexus is an evidence-driven control plane for AI applications and agents that routes and optimizes execution, verifies high-risk outputs and actions, and authorizes autonomy through versioned policies backed by inspectable, provenance-bearing evidence.

The portfolio-grade final demo

Agent proposes an action or code change → Nexus classifies risk → policy evaluates the request → verification produces deterministic and independent evidence → evidence is recorded with provenance → policy returns ALLOW / BLOCK / APPROVAL_REQUIRED → optional repair/re-verification → execution → full trace and replay. The demo should show the same workflow under success, failure and adversarial conditions.

What to avoid

- Do not claim universal safety or universal trust.
- Do not use a single scalar trust score to hide uncertainty.
- Do not call an LLM judgment 'proof'.
- Do not add infrastructure because it looks impressive; add it when a measured requirement justifies it.
- Do not build every subsystem deeply at the same time.
- Do not sacrifice explainability for feature count.